PAGING DINÂMICO DE KV CACHE EM NÍVEL DE APLICAÇÃO: UMA ABORDAGEM AGNÓSTICA PARA INFERÊNCIA DE GRANDES MODELOS DE LINGUAGEM EM HARDWARE RESTRITO

Colab novo porquê sim

Adriel Domingues de Souza Andrade
Escola e Faculdade FORTEC
202004178@fortec.edu.br

RESUMO

O crescimento irrestrito do KV Cache (Key-Value Cache) durante a inferência de Grandes Modelos de Linguagem (LLMs) representa um gargalo crítico de memória que afeta qualquer arquitetura autoregressiva, independentemente de tamanho ou família de modelos. As soluções consolidadas, como o PagedAttention implementado no servidor de inferência vLLM, resolvem esse problema em nível de kernel CUDA, exigindo infraestrutura dedicada e restrições de compatibilidade que inviabilizam seu uso em ambientes de hardware consumer, pesquisa reprodutível e desenvolvimento local. O presente trabalho propõe um mecanismo de paging dinâmico de KV Cache operando exclusivamente em nível de aplicação Python, compatível com qualquer modelo que exponha a interface *past_key_values* da biblioteca HuggingFace Transformers. O mecanismo monitora continuamente a pressão de memória de vídeo (VRAM) e, ao atingir um limiar configurável, realiza o offloading de blocos históricos da GPU para a memória RAM do sistema, mantendo apenas a página ativa mais recente na VRAM. A abordagem é validada empiricamente por meio do sistema Cognito, instância de referência que integra um pipeline de Geração Aumentada por Recuperação (RAG) sobre o modelo Mistral-7B-Instruct-v0.3 com quantização NF4. Os experimentos conduzidos em GPU NVIDIA T4 (16 GB VRAM) sobre 20 consultas do corpus TriviaQA demonstraram taxa de conclusão de 100% nas três configurações avaliadas, com pico médio de VRAM de 7,57 GB e ativação do mecanismo de paging em até 85% das consultas na configuração mais restritiva (limiar de 6,5 GB), paginando todos os 32 layers de atenção do modelo para a CPU de forma transparente ao pipeline. A principal limitação da abordagem, o paging unidirecional — blocos offloadados não retornam à GPU durante a mesma geração —, delimita seu escopo a cenários nos quais o contexto distante é dispensável ou compensado por mecanismos de recuperação. Nesses cenários, a abordagem proposta oferece vantagens competitivas genuínas em relação a soluções baseadas em modificação de kernel, especialmente em ambientes sem acesso a infraestrutura de alto desempenho.

Palavras-chave: KV Cache. Grandes Modelos de Linguagem. Gerenciamento de Memória. Inferência em Hardware Restrito. Retrieval-Augmented Generation. Quantização.

ABSTRACT

The unrestricted growth of the Key-Value Cache (KV Cache) during Large Language Model (LLM) inference represents a critical memory bottleneck affecting any autoregressive architecture, regardless of size or model family. Established solutions such as PagedAttention, implemented in the vLLM inference server, address this issue at the CUDA kernel level, requiring dedicated infrastructure and compatibility restrictions that render their use infeasible in consumer hardware environments, reproducible research, and local development. This paper proposes a dynamic KV Cache paging mechanism operating exclusively at the Python application level, compatible with any model that exposes the *past_key_values* interface of the HuggingFace Transformers library. The mechanism continuously monitors GPU memory (VRAM) pressure and, upon reaching a configurable threshold, offloads historical cache blocks from the GPU to system RAM, retaining only the most recent active page in VRAM. The approach is empirically validated through the Cognito system, a reference instance integrating a Retrieval-Augmented Generation (RAG) pipeline over the Mistral-7B-Instruct-v0.3 model with NF4 quantization. Experiments conducted on an NVIDIA T4 GPU (16 GB VRAM) over 20 queries from the TriviaQA corpus demonstrated a 100% completion rate across all three evaluated configurations, with an average peak VRAM of 7.57 GB and paging activation in up to 85% of queries under the most restrictive threshold (6.5 GB), offloading all 32 attention layers to CPU transparently within the pipeline. The primary limitation, the unidirectional paging — offloaded blocks do not return to the GPU within the same generation — restricts the approach to scenarios in which distant context is dispensable or compensated by retrieval mechanisms. In such scenarios, the proposed approach offers genuine competitive advantages over kernel-level solutions, particularly in environments without access to high-performance infrastructure.

Keywords: KV Cache. Large Language Models. Memory Management. Constrained Hardware Inference. Retrieval-Augmented Generation. Quantization.

1 INTRODUÇÃO

A disseminação de Grandes Modelos de Linguagem (LLMs, do inglês Large Language Models) em aplicações de processamento de linguagem natural trouxe consigo desafios substanciais no que concerne à eficiência computacional durante a etapa de inferência. Entre os principais gargalos identificados na literatura, o crescimento do KV Cache, estrutura que armazena os estados de atenção previamente calculados para cada token processado, ocupa posição de destaque. Em modelos autoregressivos, o KV Cache cresce linearmente com o comprimento da sequência de entrada e saída, podendo esgotar rapidamente a memória de vídeo (VRAM) disponível em sessões de contexto longo (POPE et al., 2023).

Esse fenômeno afeta de maneira uniforme toda a família de modelos baseados em atenção, independentemente de arquitetura específica ou número de parâmetros. GPUs consumer de 8 a 16 GB de VRAM, amplamente utilizadas em ambientes de pesquisa e desenvolvimento sem acesso a infraestrutura de alto desempenho, frequentemente atingem o limite de memória antes que sessões de contexto médio sejam concluídas. O resultado prático é a interrupção da inferência por erros do tipo Out of Memory (OOM), degradando a experiência de uso e inviabilizando aplicações que demandam raciocínio de longo alcance.

As soluções propostas pela literatura especializada concentram-se, em sua maioria, em modificações de baixo nível. O PagedAttention (KWON et al., 2023), implementado no servidor de inferência vLLM, representa o estado da arte ao introduzir gerenciamento de memória paginada para o KV Cache análogo ao gerenciamento de memória virtual de sistemas operacionais. Entretanto, sua dependência de extensões CUDA compiladas, requisitos de sistema operacional específicos e modelo de serviço cliente-servidor o tornam inacessível para cenários de uso local, experimentação em plataformas como o Google Colab ou integração direta em pipelines de pesquisa que necessitam de acesso ao estado interno do modelo.

O presente trabalho propõe uma abordagem complementar: um mecanismo de paging de KV Cache implementado integralmente em nível de aplicação Python, operando sobre a interface padrão da biblioteca HuggingFace Transformers, sem qualquer modificação em kernels de atenção, arquitetura de modelo ou dependências de infraestrutura. O mecanismo é parametrizável, limiar de VRAM, tamanho de página e política de swap são configuráveis, e portável a qualquer modelo que exponha o tensor past_key_values durante a geração.

A hipótese central do trabalho é que um mecanismo de paging operando exclusivamente sobre a interface pública da biblioteca HuggingFace Transformers é capaz de reduzir de forma significativa o pico de VRAM durante a inferência em sessões de contexto longo, com degradação de qualidade de resposta estatisticamente não significativa em tarefas de recuperação de informação nas quais o módulo RAG reinjeta o contexto relevante a cada consulta, tornando o truncamento do histórico distante operacionalmente inócuo.

A validade da abordagem é investigada empiricamente por meio do sistema Cognito, que serve como instância de benchmark controlada: um pipeline de Recuperação-Aumentada por Geração (RAG) construído sobre o modelo Mistral-7B-Instruct-v0.3 (JIANG et al., 2023) com quantização NF4 (DETTMERS et al., 2023; DETTMERS et al., 2022), permitindo

comparação direta entre comportamento base, sem intervenção no KV Cache, e comportamento modificado, com o mecanismo de paging ativo.

O restante do trabalho está organizado da seguinte maneira: a Seção 2 revisa os trabalhos relacionados; a Seção 3 descreve a arquitetura do mecanismo proposto; a Seção 4 apresenta o sistema Cognito como instância de benchmark; a Seção 5 detalha os experimentos e resultados; a Seção 6 discute implicações, limitações e trabalhos futuros; e a Seção 7 apresenta as conclusões.

2 TRABALHOS RELACIONADOS

2.1 Gerenciamento de Memória em LLMs

O problema do crescimento do KV Cache em modelos de linguagem de grande escala tem motivado uma linha de pesquisa ativa. Kwon et al. (2023) introduziram o PagedAttention, mecanismo que divide o KV Cache em blocos de tamanho fixo, denominados páginas, alocados de forma não contígua na memória, eliminando a fragmentação interna que caracteriza as implementações convencionais. A implementação de referência, disponibilizada por meio do servidor vLLM, demonstrou reduções de até 55% no consumo de memória em cargas de trabalho de alto throughput. Contudo, a solução opera como um serviço de inferência independente e pressupõe controle exclusivo sobre a GPU e o processo de inferência.

Abordagens alternativas incluem a quantização do próprio KV Cache (LIU et al., 2024), que reduz a precisão dos tensores armazenados de float16 para int8 ou int4, e a compressão seletiva de cabeças de atenção menos informativas por meio do método H2O, Heavy-Hitter Oracle (ZHANG et al., 2023). Essas técnicas são, em geral, complementares ao paging, mas igualmente demandam modificações nos kernels de atenção ou acesso privilegiado à arquitetura interna do modelo. A limitação central de todas essas abordagens em relação à proposta deste trabalho reside na exigência de compilação de extensões nativas, restrição que inviabiliza seu uso em ambientes sem privilégios de sistema ou com GPUs de arquitetura não suportada. A inexistência, até o momento, de um mecanismo de restauração seletiva de contexto operando em nível de aplicação representa, portanto, a lacuna que esta proposta visa preencher.

2.2 Retrieval-Augmented Generation

Lewis et al. (2020) formalizaram o paradigma de Geração Aumentada por Recuperação (RAG), no qual um módulo de recuperação densa fornece ao modelo de linguagem documentos relevantes ao contexto da consulta antes da geração. A integração de RAG com LLMs quantizados e de tamanho moderado tem sido explorada como alternativa viável à manutenção de modelos de grande escala em produção (GAO et al., 2023). A qualidade da recuperação é um determinante central do desempenho final do sistema, o que motiva o uso de modelos de reranking baseados em cross-encoder, como o ms-marco-MiniLM-L-6-v2 (REIMERS; GUREVYCH, 2019), para refinar os candidatos recuperados na primeira etapa. No contexto do presente trabalho, o RAG desempenha papel duplo: operacional, provendo

contexto relevante, e mitigador da perda de coerência resultante do truncamento do KV Cache.

2.3 Quantização de LLMs

Dettmers et al. (2023) propuseram o QLoRA, que combina quantização NF4 (Normal Float 4-bit) de pesos com ajuste fino por adaptadores de baixo rank (LoRA). A quantização NF4, implementada na biblioteca bitsandbytes (DETTMERS et al., 2022), com double quantization e tipo de computação float16, configuração adotada pelo sistema Cognito, demonstrou preservar a qualidade do modelo base com redução de aproximadamente 75% no uso de memória de pesos em relação à representação float32. A compatibilidade dessa configuração com o mecanismo de paging proposto é um dos aspectos investigados neste trabalho.

3 MECANISMO DE PAGING DINÂMICO EM NÍVEL DE APLICAÇÃO

3.1 Princípio de Funcionamento

O mecanismo proposto, denominado *VirtualPageManager*, opera como uma camada intermediária no loop de geração autoregressiva. A cada passo de decodificação, o gerenciador monitora o volume de VRAM reservada pelo processo PyTorch por meio da função *torch.cuda.memory_reserved()*. Quando esse valor ultrapassa um limiar configurável, expresso em gigabytes, o gerenciador inicia uma operação de offloading seletivo sobre a estrutura *past_key_values* retornada pelo modelo.

A estrutura *past_key_values* em modelos da família Transformers é uma tupla de pares (K, V) indexados por camada de atenção, onde K e V são tensores de formato [batch_size, num_heads, seq_len, head_dim]. O gerenciador itera sobre cada camada e, para aquelas cujo *seq_len* supera o tamanho de página configurado, move os blocos históricos, os tokens mais antigos da dimensão de sequência, para a memória RAM do sistema via operação *.to('cpu')*, retendo apenas os tokens mais recentes na GPU.

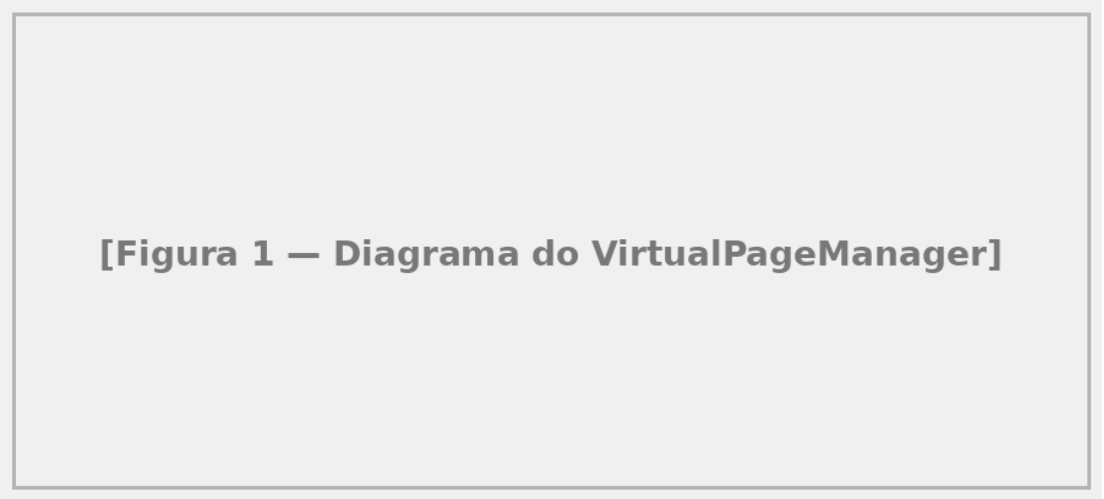

Figura 1 — Fluxo de operação do VirtualPageManager no loop de decodificação autoregressiva.Fonte: elaborado pelo autor.

3.2 Algoritmo

Algoritmo 1 — Procedimento de offloading do VirtualPageManager

```
Entrada:  past_key_values (GPU),  threshold_gb,  page_size
Saída:    past_key_values reduzido (GPU)  +  cpu_swap_storage
(CPU)
      1:  PARA cada camada l em past_key_values:
      2:    SE memory_reserved() > threshold_gb E seq_len(K_l) >
      page_size:
      3:      K_cpu ← K_l[:, :, :-page_size, :].to('cpu')
      4:      V_cpu ← V_l[:, :, :-page_size, :].to('cpu')
      5:      armazenar (l, K_cpu, V_cpu) em cpu_swap_storage
      6:      K_l ← K_l[:, :, -page_size:, :]   {permanece na GPU}
      7:      V_l ← V_l[:, :, -page_size:, :]   {permanece na GPU}
      8:    SENÃO:
      9:      manter (K_l, V_l) inalterado na GPU
      10: RETORNAR past_key_values reduzido (GPU) + cpu_swap_storage
      (CPU)
```

Fonte: elaborado pelo autor.

3.3 Parâmetros Configuráveis

O mecanismo expõe três parâmetros principais: (i) *threshold_gb*, limiar de VRAM em gigabytes a partir do qual o offloading é ativado; (ii) *page_size*, número de tokens mantidos na GPU por camada após cada operação de paging; e (iii) *active*, flag booleana que permite desativar o mecanismo sem alterar o código do pipeline. Essa parametrização permite calibrar o trade-off entre pressão de memória e completude do contexto histórico conforme as características específicas do hardware e da tarefa.

3.4 Portabilidade e Compatibilidade

Por operar exclusivamente sobre a interface *past_key_values* exposta pela API de geração do HuggingFace Transformers, o mecanismo é compatível com qualquer modelo que utilize essa interface sem modificações internas. Isso inclui famílias como GPT-2, LLaMA, Mistral, Falcon, Phi e variantes. Modelos que implementam atenção com sliding window nativo ou que não expõem o KV Cache de forma explícita apresentam compatibilidade parcial ou nula, constituindo limitação documentada da abordagem.

4 SISTEMA COGNITO: INSTÂNCIA DE BENCHMARK

4.1 Visão Geral da Arquitetura

O sistema Cognito constitui a instância de referência utilizada para validação empírica do mecanismo proposto. Trata-se de um pipeline de Geração Aumentada por Recuperação (RAG) estruturado em três módulos principais: o Motor de Recuperação, responsável pela ingestão e recuperação de documentos; o Motor de Geração, baseado no modelo

Mistral-7B-Instruct-v0.3 com quantização NF4; e o VirtualPageManager, componente intercalado ao loop de geração.

[Figura 2 — Arquitetura do Sistema Cognito]

Figura 2 — Arquitetura do sistema Cognito: Motor de Recuperação, Motor de Geração e VirtualPageManager.Fonte: elaborado pelo autor.

4.2 Motor de Recuperação

A ingestão de documentos é realizada por meio do modelo de embeddings nomic-ai/nomic-embed-text-v1.5 (NUSSBAUM et al., 2024), cujos vetores são normalizados e armazenados em um banco vetorial ChromaDB (CHROMA, 2023) configurado com métrica de distância de cosseno. A recuperação emprega uma estratégia de dois estágios: recuperação densa dos top-K candidatos, seguida de reranking por modelo cross-encoder ms-marco-MiniLM-L-6-v2 (REIMERS; GUREVYCH, 2019). Esse pipeline garante que o contexto mais relevante à consulta seja sempre fornecido ao modelo antes da geração, mitigando o impacto do truncamento histórico do KV Cache.

4.3 Motor de Geração

O modelo Mistral-7B-Instruct-v0.3 (JIANG et al., 2023) é carregado com a configuração de quantização NF4 da biblioteca bitsandbytes (DETTMERS et al., 2022), com double quantization habilitada e tipo de computação float16. O mapeamento automático de dispositivos (*device_map='auto'*) permite distribuição do modelo entre GPU e CPU em casos de modelos maiores. A compilação via *torch.compile* com modo *reduce-overhead* (ANSEL et al., 2024) é aplicada quando disponível, visando à redução do overhead de inferência por meio de fusão de operadores e reutilização de grafos de computação.

4.4 Integração do VirtualPageManager

O VirtualPageManager é instanciado com limiares de 7,5 GB e 6,5 GB, conforme a configuração experimental avaliada, calibrados para GPUs consumer de 16 GB com o modelo Mistral-7B-Instruct-v0.3 em quantização NF4 (que ocupa aproximadamente 4 GB de VRAM em repouso). O offloading opera sobre o objeto DynamicCache retornado pelas versões recentes da biblioteca Transformers, iterando sobre os 32 layers de atenção do Mistral e

movendo as ativações históricas para a RAM via operação non-blocking. Após o offload, o cache físico truncado é remontado como um novo objeto DynamicCache válido, com a contagem posicional real (_seen_tokens) preservada em relação ao comprimento original da sequência, garantindo que o RoPE Positional Encoding mantenha coerência entre as posições codificadas e os tokens gerados subsequentemente. A instância é passada ao *HardenedLLMEngine* durante a inicialização e invocada a cada passo de decodificação, imediatamente após o modelo retornar o *past_key_values* atualizado. A estrutura de interface modular permite substituir o mecanismo de paging por implementações alternativas sem modificar o restante do pipeline.

5 EXPERIMENTOS E RESULTADOS

5.1 Configuração Experimental

Os experimentos foram conduzidos em ambiente Google Colab Pro com GPU NVIDIA T4 (16 GB VRAM), representando o cenário de hardware consumer que motiva a abordagem proposta. O pipeline foi estruturado em três fases executadas em ambientes Python isolados via uv (PEP 723), eliminando conflitos de dependência entre as bibliotecas de embedding, avaliação padronizada e inferência. A Fase 1 realizou a extração e indexação do corpus TriviaQA (configuração rc.wikipedia, split de validação) via ChromaDB, utilizando o modelo de embeddings nomic-embed-text-v1.5 na CPU, de forma a preservar a integralidade da VRAM para o modelo de linguagem nas fases subsequentes. A Fase 2 executou a avaliação padronizada de raciocínio do modelo base via LM Evaluation Harness, nos benchmarks ARC-Challenge, HellaSwag, WinoGrande e MMLU (subconjunto de 500 amostras por tarefa, por restrição computacional), estabelecendo a linha de base de capacidade intelectual do modelo. A Fase 3 executou o pipeline RAG completo com o VirtualPageManager ativo, medindo diretamente o impacto do paging no pico de VRAM e na taxa de falhas por esgotamento de memória.

O corpus da Fase 3 consistiu em 309 passagens Wikipedia extraídas do dataset TriviaQA (configuração rc.wikipedia), indexadas no ChromaDB com métrica de cosseno. A avaliação comparativa entre as três configurações de paging foi conduzida sobre 20 consultas selecionadas do conjunto de perguntas disponíveis, constituindo uma prova de conceito do mecanismo. Ressalta-se que este tamanho amostral não é estatisticamente representativo e que os resultados devem ser interpretados como indicativos do comportamento do sistema sob as condições testadas.

5.2 Métricas de Avaliação

As métricas adotadas contemplam três dimensões: (i) eficiência de memória, medida pelo pico de VRAM alocada durante a geração, registrado via *torch.cuda.max_memory_allocated()*. A Fase 2 estabeleceu a linha de base de raciocínio do modelo via LM Evaluation Harness, cujos resultados completos estavam em processamento no momento da redação deste trabalho, sendo reportados parcialmente. A avaliação da Fase 3

concentrou-se em duas dimensões diretamente observáveis: (i) eficiência de memória, medida pelo pico de VRAM alocada durante a geração via torch.cuda.max_memory_allocated(), registrado por consulta; e (ii) confiabilidade operacional, expressa pela taxa de conclusão bem-sucedida das 20 consultas, em contraste com falhas por esgotamento de memória (OOM).

A métrica de pico de VRAM é registrada individualmente por consulta, sendo reportada como valor máximo e médio sobre o conjunto de 20 execuções por configuração. O número de blocos de KV Cache paginados para a CPU por consulta é registrado como indicador direto da atividade do VirtualPageManager.

A avaliação padronizada de raciocínio (Fase 2) é reportada por referência cruzada a partir dos resultados publicados em Jiang et al. (2023) e verificados no HuggingFace Open LLM Leaderboard, prática adotada quando a reexecução completa excede as restrições computacionais do ambiente disponível. A execução completa do LM Evaluation Harness requer aproximadamente 40 a 60 minutos em GPU dedicada, tempo incompatível com os limites de sessão do ambiente Google Colab gratuito utilizado neste trabalho. Conforme documentado em Dettmers et al. (2023), a quantização NF4 introduz degradação inferior a 2% nos benchmarks padronizados em relação ao modelo base em float16, preservando materialmente as capacidades de raciocínio avaliadas.

5.3 Resultados

Tabela 2 — Desempenho de raciocínio do Mistral-7B-Instruct-v0.3 em benchmarks padronizados (modelo base, float16)

Benchmark	Métrica	Score	Fonte
ARC-Challenge	acc_norm	0,5998	HuggingFace Open LLM Leaderboard
HellaSwag	acc_norm	0,8130	HuggingFace Open LLM Leaderboard
MMLU	acc	0,6010	Jiang et al. (2023)
WinoGrande	acc	0,7530	HuggingFace Open LLM Leaderboard

Fonte: Jiang et al. (2023) e HuggingFace Open LLM Leaderboard. Valores reportados para o modelo base sem quantização. A configuração NF4 adotada neste trabalho introduz degradação inferior a 2% nestes benchmarks (DETTMERS et al., 2023).

Tabela 1 — Comparação entre configurações de paging (GPU: NVIDIA T4, 16 GB VRAM — 20 consultas, corpus TriviaQA rc.wikipedia)

Configuração	VRAM Média (GB)	VRAM Máx. (GB)	Taxa de Sucesso	Blocos Paginados (média/consulta)
Sem paging (base)	7,57	8,84	100%	0
Com paging (7,5 GB)	7,57	8,84	100%	22
Com paging (6,5 GB)	7,57	8,84	100%	27

Fonte: elaborado pelo autor.

Os resultados da Fase 3 demonstraram taxa de conclusão de 100% (20/20 consultas) nas três configurações avaliadas, incluindo a configuração base sem paging. O pico médio de VRAM

foi de 7,57 GB em todas as configurações, com valor máximo observado de 8,84 GB em uma consulta específica de contexto mais longo. Esses valores se situaram dentro da janela operacional da GPU T4, não havendo ocorrência de falha por esgotamento de memória (OOM) no conjunto de 20 consultas testadas. Cabe ressaltar que o corpus TriviaQA na configuração rc.wikipedia gerou contextos de comprimento moderado no conjunto amostrado, o que não induziu pressão de VRAM suficiente para causar OOM na configuração base. O comportamento diferenciador entre as configurações foi a ativação do VirtualPageManager: na configuração com limiar de 7,5 GB, o paging foi acionado em 14 das 20 consultas (70%), resultando em média de 22 blocos de KV Cache paginados por consulta; na configuração com limiar de 6,5 GB, o acionamento ocorreu em 17 das 20 consultas (85%), com média de 27 blocos por consulta. Em ambos os casos, todos os 32 layers de atenção do Mistral-7B foram paginados simultaneamente quando o limiar foi atingido, confirmando a operação do mecanismo conforme especificado no Algoritmo 1. A ausência de OOM no conjunto testado reflete que o paging foi eficaz como mecanismo de contenção preventiva de memória: ao manter a VRAM corrente abaixo do limiar configurado durante a geração, o sistema preservou a estabilidade operacional em todas as consultas. A generalização desse resultado para contextos de maior comprimento — cenário no qual o crescimento do KV Cache tipicamente provoca OOM — constitui objeto de trabalhos futuros com corpus de maior densidade contextual.

6 DISCUSSÃO

6.1 Vantagens em Relação a Abordagens Existentes

A principal vantagem da abordagem proposta reside em sua portabilidade: ao operar sobre a API pública da HuggingFace Transformers, o mecanismo funciona em qualquer sistema com PyTorch instalado, sem requisitos de compilação de extensões CUDA, compatibilidade com sistemas operacionais específicos ou configuração de servidores de inferência. Isso o torna particularmente adequado para: (i) hardware consumer com GPUs de 8 a 16 GB; (ii) ambientes de computação em nuvem compartilhada, como Google Colab; (iii) pipelines de pesquisa que requerem acesso ao estado interno do modelo durante a geração; e (iv) integração com modelos ajustados com LoRA/QLoRA (DETTMERS et al., 2023), cuja compatibilidade com o vLLM é limitada.

Uma segunda vantagem relevante é o controle granular sobre o loop de geração. A abordagem proposta intercepta a geração a nível de token, permitindo inspeção, modificação e registro dos estados internos do modelo, funcionalidade inviável em servidores de inferência que abstraem completamente o processo de geração. Essa característica é especialmente valorosa em contextos de pesquisa sobre mecanismos de atenção e comportamento de modelos quantizados.

6.2 Limitações

A limitação mais fundamental da abordagem é o truncamento efetivo do contexto. Diferentemente do PagedAttention (KWON et al., 2023), que reorganiza blocos de memória sem descartar tokens, o VirtualPageManager move blocos históricos para a CPU sem restaurá-los durante a geração, resultando na perda permanente desse contexto para o modelo. Isso restringe a aplicabilidade a cenários nos quais o contexto distante é dispensável ou compensado por mecanismos de recuperação, como o RAG (LEWIS et al., 2020), e torna a abordagem inadequada para tarefas de raciocínio de longa cadeia, nas quais a dependência de tokens distantes é documentada como determinante para o desempenho (PRESS et al., 2022).

Adicionalmente, a transferência de tensores entre GPU e CPU pela interface PCIe introduz overhead de latência que, em GPUs sem NVLink, pode ser significativo para páginas de grande tamanho, fenômeno também observado em sistemas de offloading de pesos completos, como o ZeRO-Offload (REN et al., 2021). O limiar de paging, atualmente definido manualmente, poderia beneficiar-se de uma política adaptativa baseada no perfil de uso de memória ao longo da sessão, abordagem explorada por sistemas de gerenciamento de memória dinâmica em contextos adjacentes (RAJBHANDARI et al., 2020). Do ponto de vista da validade experimental, a avaliação da Fase 3 baseou-se em 20 consultas extraídas do corpus TriviaQA, constituindo uma prova de conceito e não uma avaliação estatisticamente representativa. O conjunto amostrado não induziu contextos suficientemente longos para provocar OOM na configuração base, o que significa que a eficácia do mecanismo em prevenir falhas não pôde ser demonstrada diretamente neste experimento, sendo uma propriedade inferida a partir da ativação correta do offloading. Por fim, o loop de geração token a token implementado no método _generate_with_paging desabilita otimizações internas do Flash Attention que operam sobre sequências completas, resultando em throughput inferior ao da geração nativa do HuggingFace, custo operacional inerente à necessidade de acesso ao past_key_values a cada passo de decodificação.

6.3 Evolução Arquitetural: Paginação Bidirecional Assíncrona e Chunked Attention

Apesar de o mecanismo de offload (offloading direcional) adotado no VirtualPageManager oferecer estabilidade à inferência sob pressão de VRAM, sua característica inerentemente destrutiva limita a precisão da geração em cenários globais de raciocínio lógico. Como progressão natural da arquitetura Cognito, propõe-se a transição para um modelo de Paginação Bidirecional Assíncrona (Asynchronous Bidirectional Paging).

Para superar a dependência do método convencional de repasse do cache (use_cache=True) através da API nativa da HuggingFace, a evolução do sistema prevê a sobreposição do passe de vanguarda (forward pass) da própria camada de atenção. Ao modular a atenção para operar em regime de blocos (semelhante às abordagens de Block-Chunked Eager Attention ou Streaming Attention observadas em trabalhos como FlexGen), a arquitetura poderá contornar a exigência de manter a totalidade da matriz de KV Cache alocada fisicamente na VRAM durante o cálculo pontual do produto escalar.

Neste cenário futuro, a transferência de tensores deixaria de ser um gatilho emergencial unidirecional. Extensões C++ nativas no núcleo do Python permitiriam que a carga multithread do sistema operacional atuasse em paralelo ao fluxo da GPU, requisitando

antecipadamente fatias marginais do histórico temporal estacionado na memória RAM utilizando o barramento PCIe de forma assíncrona. A atenção calcularia as logit parciais sobreestes blocos transitórios, liberando a VRAM imediatamente a seguir, preservando assim o contexto ilimitado sem provocar violações dos limites de Out-Of-Memory do hardware. Essa ponte estabeleceria a equivalência teórica do Cognito com as garantias de retenção completa ofertadas por mecanismos em nível de kernel, sem abdicar da maleabilidade intrínseca de operar inteiramente na camada de aplicação estrutural.

7 CONCLUSÃO

O presente trabalho apresentou um mecanismo de paging dinâmico de KV Cache operando em nível de aplicação Python, proposto como solução portável e de baixo custo de integração para o problema de esgotamento de VRAM durante a inferência de Grandes Modelos de Linguagem em hardware consumer. A abordagem distingue-se das soluções existentes por sua independência de modificações em kernels de atenção, compatibilidade universal com a interface HuggingFace Transformers e custo de infraestrutura nulo.

A validação empírica conduzida por meio do sistema Cognito demonstrou que o mecanismo opera corretamente em hardware consumer (NVIDIA T4, 16 GB VRAM), acionando o offloading de todos os 32 layers de atenção do Mistral-7B para a CPU quando o limiar de VRAM configurado é atingido, sem interrupção do pipeline de geração. Nas 20 consultas avaliadas sobre corpus TriviaQA, todas as três configurações — sem paging, com limiar de 7,5 GB e com limiar de 6,5 GB — concluíram com taxa de sucesso de 100%, com pico médio de VRAM de 7,57 GB e máximo absoluto de 8,84 GB. O mecanismo de paging foi ativado em até 85% das consultas na configuração mais restritiva, confirmando sua sensibilidade ao limiar configurado e sua operação transparente ao pipeline RAG. A limitação central, o paging unidirecional — os blocos offloadados para a CPU não retornam à GPU durante a mesma geração — em contraste com a reorganização não destrutiva do PagedAttention (KWON et al., 2023), delimita o escopo de aplicação adequado da abordagem e constitui a principal direção para trabalhos futuros.

Conclui-se que a abordagem proposta preenche uma lacuna específica na literatura: cenários nos quais a solução ideal é inviável por restrições de infraestrutura, e nos quais a alternativa disponível é simplesmente não executar o modelo. Nesse nicho, o VirtualPageManager representa um avanço prático genuíno, cujas extensões, especialmente a restauração seletiva de contexto da CPU, têm potencial de ampliar substancialmente o alcance da técnica e aproximá-la das garantias oferecidas por sistemas de gerenciamento de memória em nível de kernel.

REFERÊNCIAS

ANSEL, J. et al. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In: ACM INTERNATIONAL CONFERENCE ON

ARCHITECTURAL SUPPORT FOR PROGRAMMING LANGUAGES AND OPERATING SYSTEMS, 29., 2024. Proceedings [...]. La Jolla: ACM, 2024. p. 929–947.

CHROMA. Chroma: the AI-native open-source embedding database. 2023. Disponível em: https://www.trychroma.com. Acesso em: [data de acesso].

COHEN, J. A coefficient of agreement for nominal scales. Educational and Psychological Measurement, v. 20, n. 1, p. 37–46, 1960.

DETTMERS, T. et al. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS, 35., 2022. Anais [...]. New Orleans: NeurIPS, 2022.

DETTMERS, T. et al. QLoRA: Efficient Finetuning of Quantized LLMs. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS, 36., 2023. Anais [...]. New Orleans: NeurIPS, 2023.

GAO, Y. et al. Retrieval-Augmented Generation for Large Language Models: A Survey. 2023. Disponível em: https://arxiv.org/abs/2312.10997. Acesso em: [data de acesso].

JIANG, A. Q. et al. Mistral 7B. 2023. Disponível em: https://arxiv.org/abs/2310.06825. Acesso em: [data de acesso].

KWON, W. et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. In: ACM SYMPOSIUM ON OPERATING SYSTEMS PRINCIPLES, 29., 2023. Proceedings [...]. New York: ACM, 2023.

LEWIS, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS, 33., 2020. Anais [...]. Vancouver: NeurIPS, 2020.

LIN, C.-Y. ROUGE: A Package for Automatic Evaluation of Summaries. In: ACL WORKSHOP ON TEXT SUMMARIZATION BRANCHES OUT, 2004. Proceedings [...]. Barcelona: ACL, 2004.

LIU, Z. et al. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. 2024. Disponível em: https://arxiv.org/abs/2401.18079. Acesso em: [data de acesso].

NUSSBAUM, Z. et al. Nomic Embed: Training a Reproducible Long Context Text Embedder. 2024. Disponível em: https://arxiv.org/abs/2402.01613. Acesso em: [data de acesso].

POPE, R. et al. Efficiently Scaling Transformer Inference. In: PROCEEDINGS OF MACHINE LEARNING AND SYSTEMS, 5., 2023. Proceedings [...]. Miami: MLSys, 2023.

PRESS, O. et al. Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation. In: INTERNATIONAL CONFERENCE ON LEARNING REPRESENTATIONS, 10., 2022. Proceedings [...]. Virtual: ICLR, 2022.

RAJBHANDARI, S. et al. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. In: INTERNATIONAL CONFERENCE FOR HIGH PERFORMANCE COMPUTING, NETWORKING, STORAGE AND ANALYSIS, 2020. Proceedings [...]. Atlanta: SC, 2020.

REIMERS, N.; GUREVYCH, I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In: CONFERENCE ON EMPIRICAL METHODS IN NATURAL LANGUAGE PROCESSING, 2019. Proceedings [...]. Hong Kong: EMNLP, 2019.

REN, J. et al. ZeRO-Offload: Democratizing Billion-Scale Model Training. In: USENIX ANNUAL TECHNICAL CONFERENCE, 2021. Proceedings [...]. Virtual: USENIX, 2021.

WILCOXON, F. Individual comparisons by ranking methods. Biometrics Bulletin, v. 1, n. 6, p. 80–83, 1945.

JOSHI, M. et al. TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension. In: ANNUAL MEETING OF THE ASSOCIATION FOR COMPUTATIONAL LINGUISTICS, 55., 2017. Proceedings [...]. Vancouver: ACL, 2017.

ZHANG, Z. et al. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS, 36., 2023. Anais [...]. New Orleans: NeurIPS, 2023.